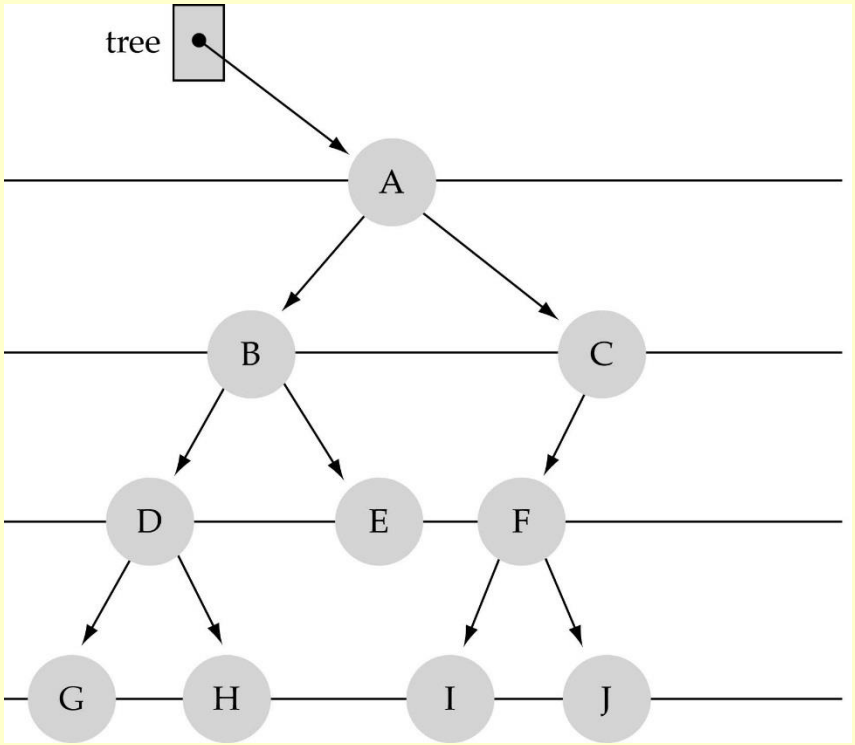
Chapter 6d
Binary Search Trees (BST)

二叉搜索树(二叉排序树)

教材9.2节

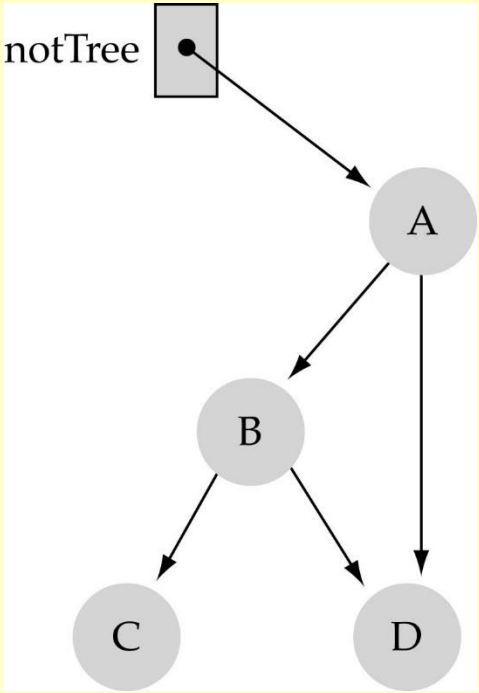
What is a binary tree二叉树?

- *Property 1*: each node can have up to two successor nodes.两个孩子结点



What is a binary tree? (cont.)

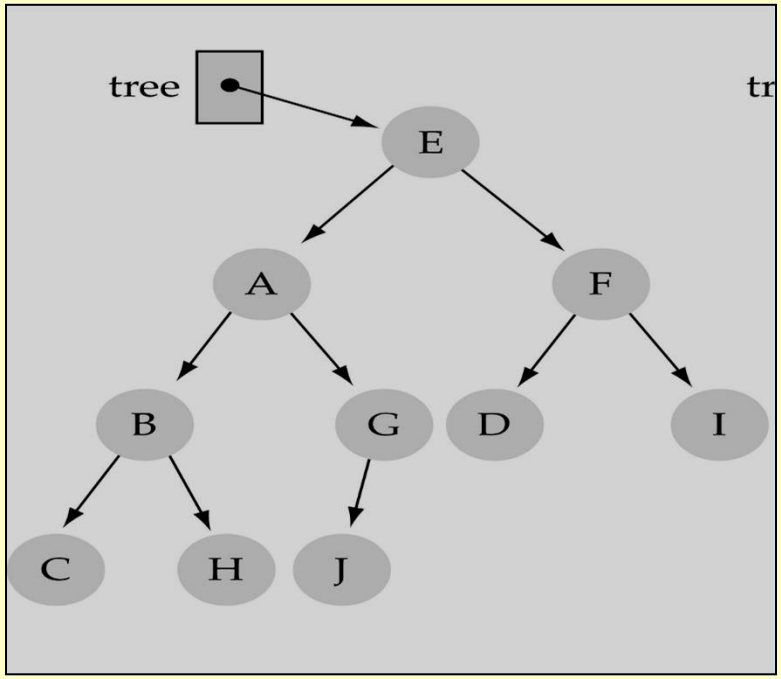
- *Property 2*: a unique path exists from the root to every other node 从根到每个其它结点只有一条路径



Not a valid binary tree!

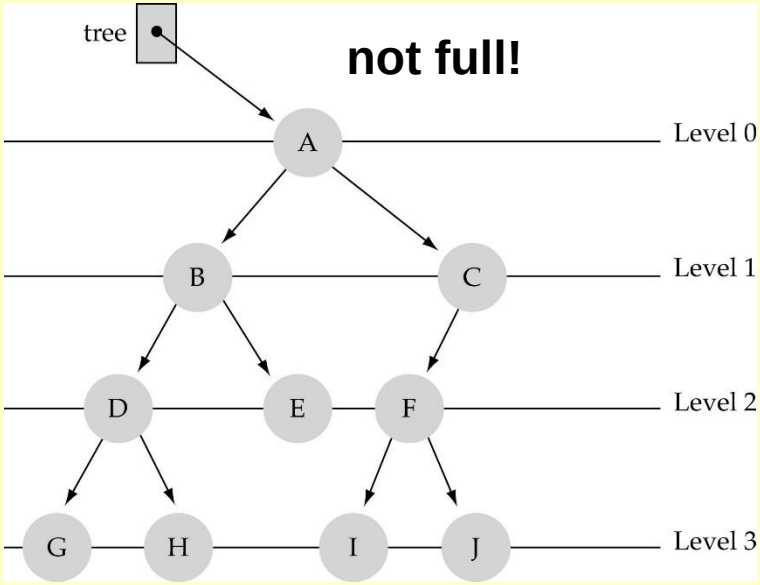
Some terminology

- The successor nodes of a node are called its *children* 孩子结点
- The predecessor node of a node is called its *parent* 父结点
- The “beginning” node is called the *root* (has no parent)根
- A node without *children* is called a *leaf* 叶子



Some terminology (cont'd)

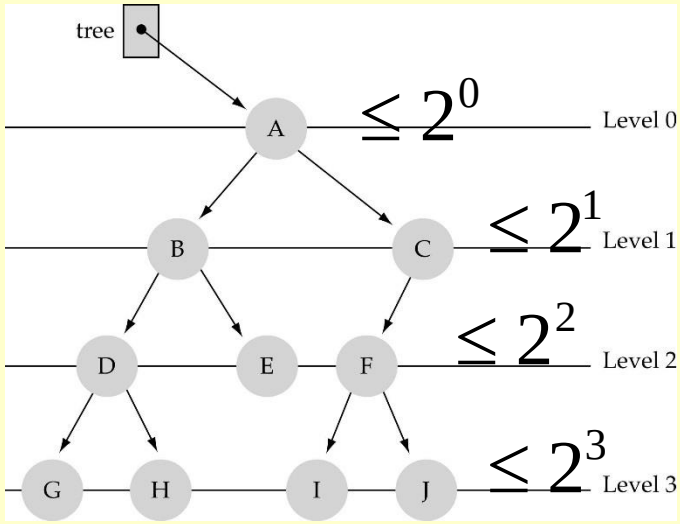
- Nodes are organize in levels (indexed from 0).
- **Level (or depth) of a node**: number of edges in the path from the root to that node.
- **Height of a tree h**: #levels = L
  (Warning: some books define h as #levels-1).
- **Full tree:** every node has exactly two children ***and*** all the leaves are on the same level.



What is the max #nodes at some level *l*?

第*l*层最多结点数

The max #nodes at level l is 2^l where $l=0,1,2,\ldots,L-1$



What is the total #nodes **N** of a full tree with height **h**? 满二叉树总结点数

$$N = \underset{l=0}{2^0} + \underset{l=1}{2^1} + \ldots + \underset{l=h-1}{2^{h-1}} = 2^h - 1$$

using the geometric series:

$$x^0 + x^1 + \ldots + x^{n-1} = \sum_{i=0}^{n-1} x^i = \frac{x^n - 1}{x - 1}$$

What is the height **h** of a full tree with **N** nodes?

$$2^h - 1 = N$$

$$\Rightarrow 2^h = N + 1$$

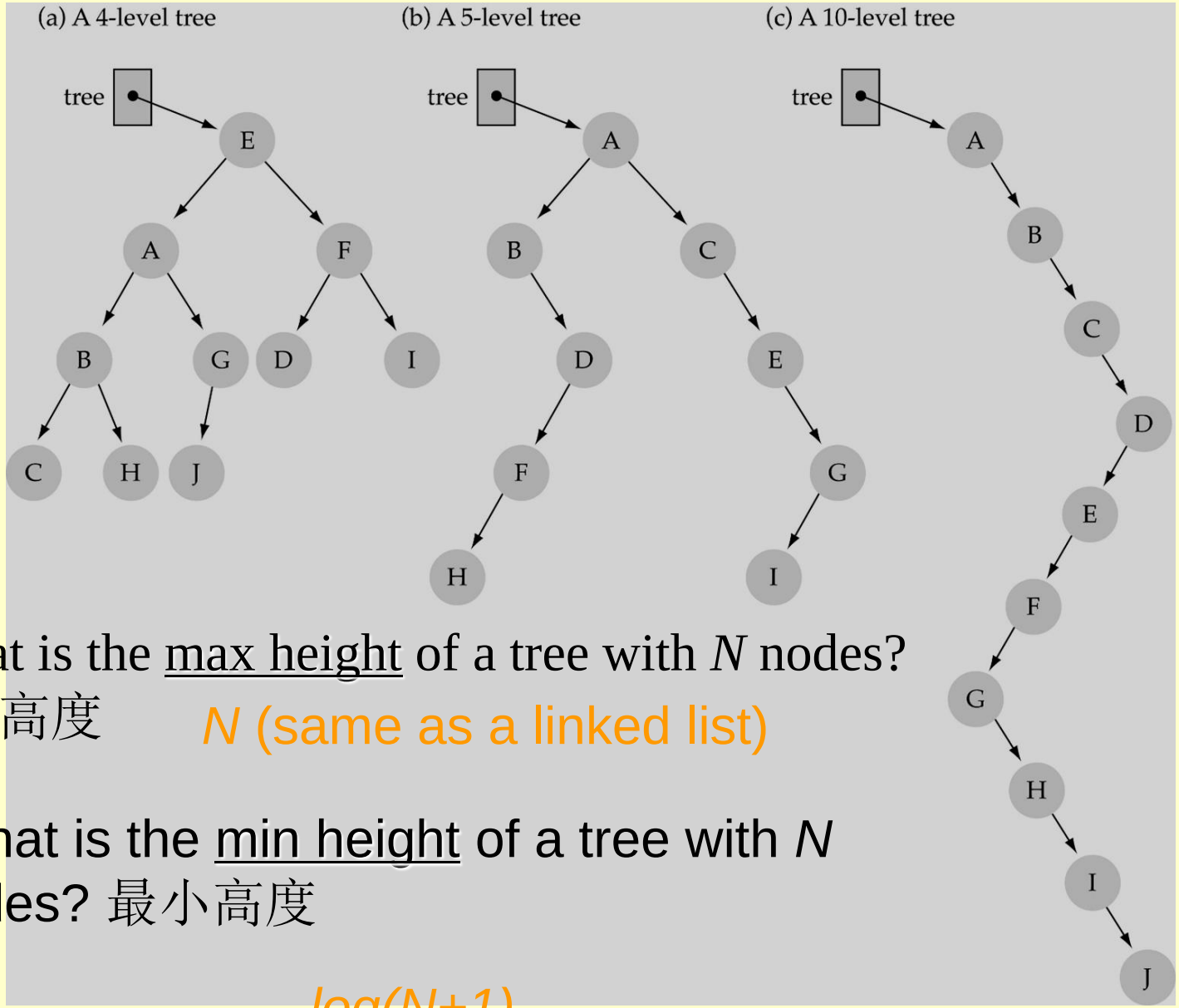
$$\Rightarrow h = \log(N+1) \rightarrow O(\log N)$$

Why is h important?

- Tree operations (e.g., insert, delete, retrieve etc.) are typically expressed in terms of h.

- So, h determines running time!

  h决定了查询效率



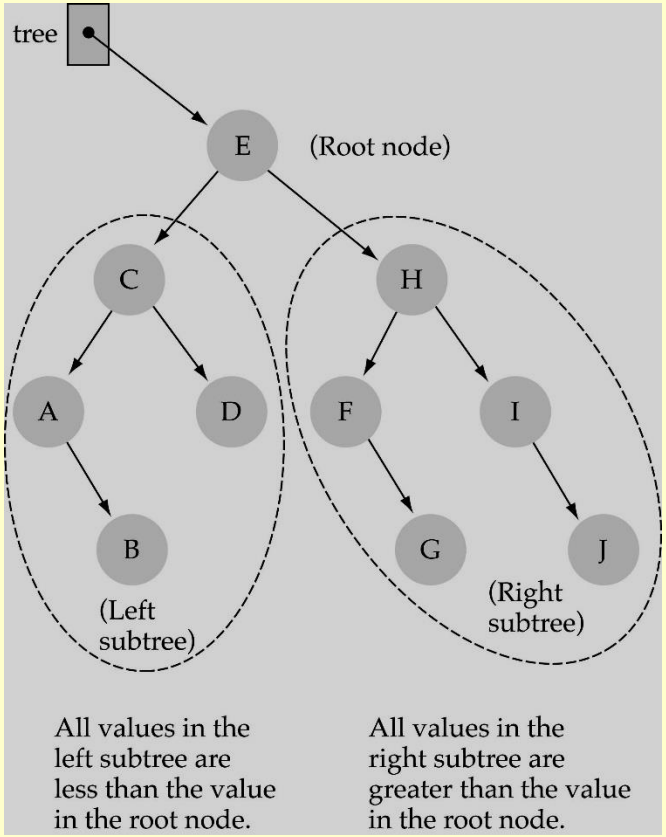
- What is the max height of a tree with *N* nodes? 最大高度 *N* (same as a linked list)

- What is the min height of a tree with *N* nodes? 最小高度 *log(N+1)*

Binary Search Trees (BSTs)

- **Binary Search Tree Property性质:**

  The value stored at a node is *greater* than the value stored at its left child and *less* than the value stored at its right child



Binary Search Trees (BSTs)

Where is the smallest element?最小元素在哪？

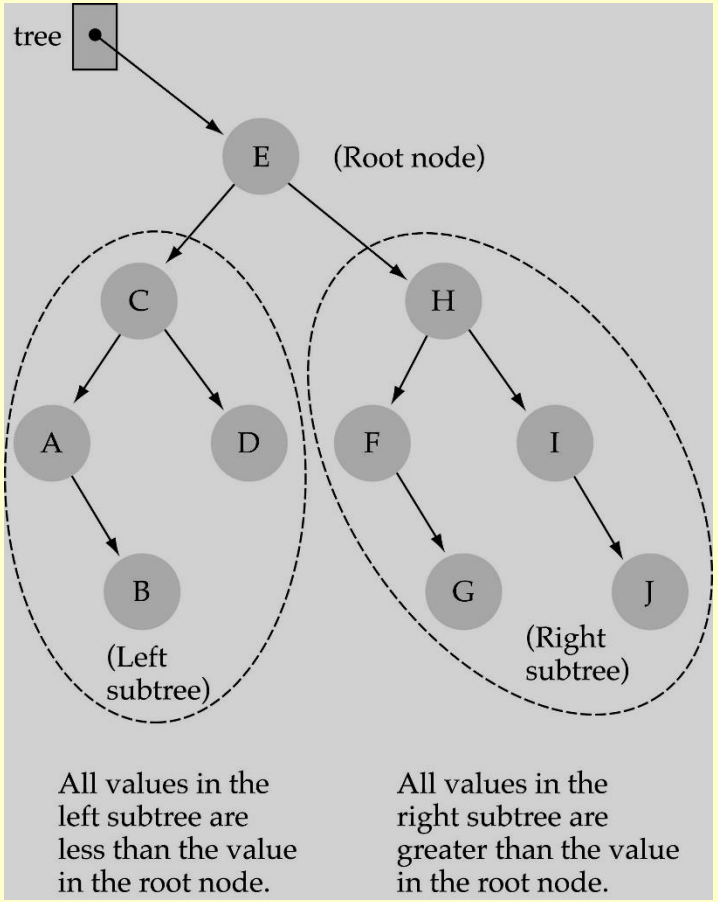
Ans: leftmost element

最左元素

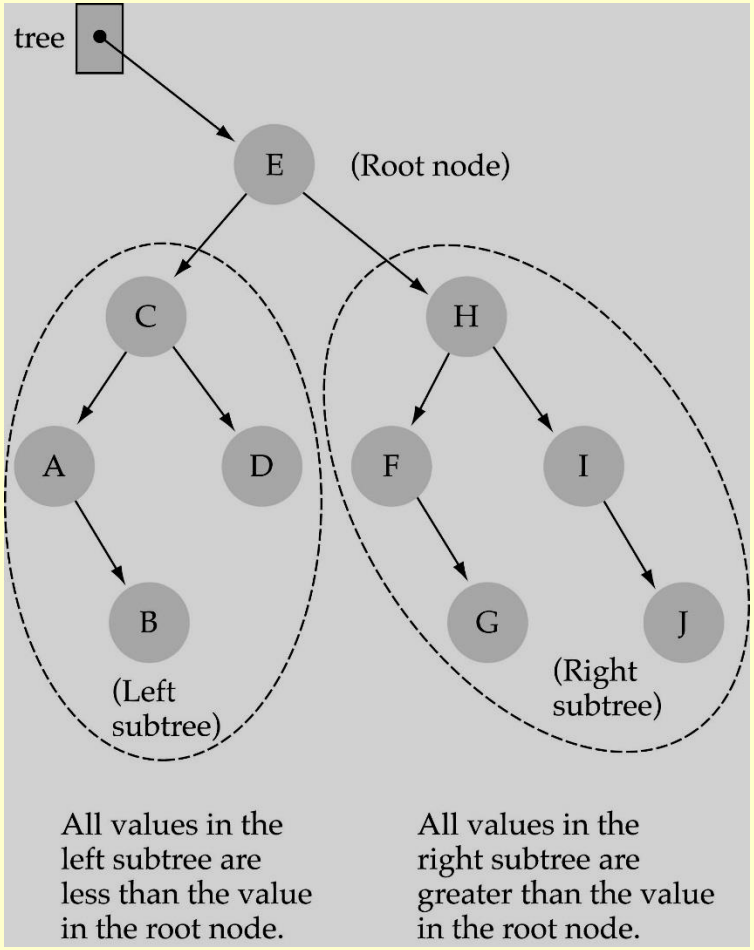
Where is the largest element?最大元素在哪?

Ans: rightmost element

最右元素



How to search a binary search tree?
查询二叉搜索树

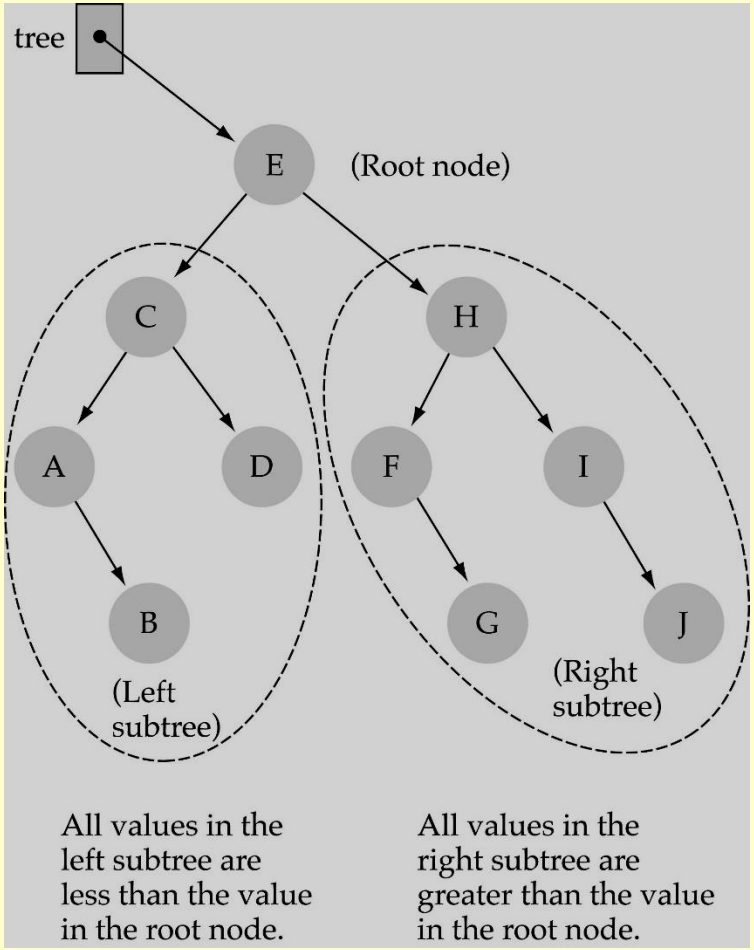


(1) Start at the root

(2) Compare the value of the item you are searching for with the value stored at the root

(3) If the values are equal, then *item found*; otherwise, if it is a leaf node, then *not found*

How to search a binary search tree?
查询二叉搜索树

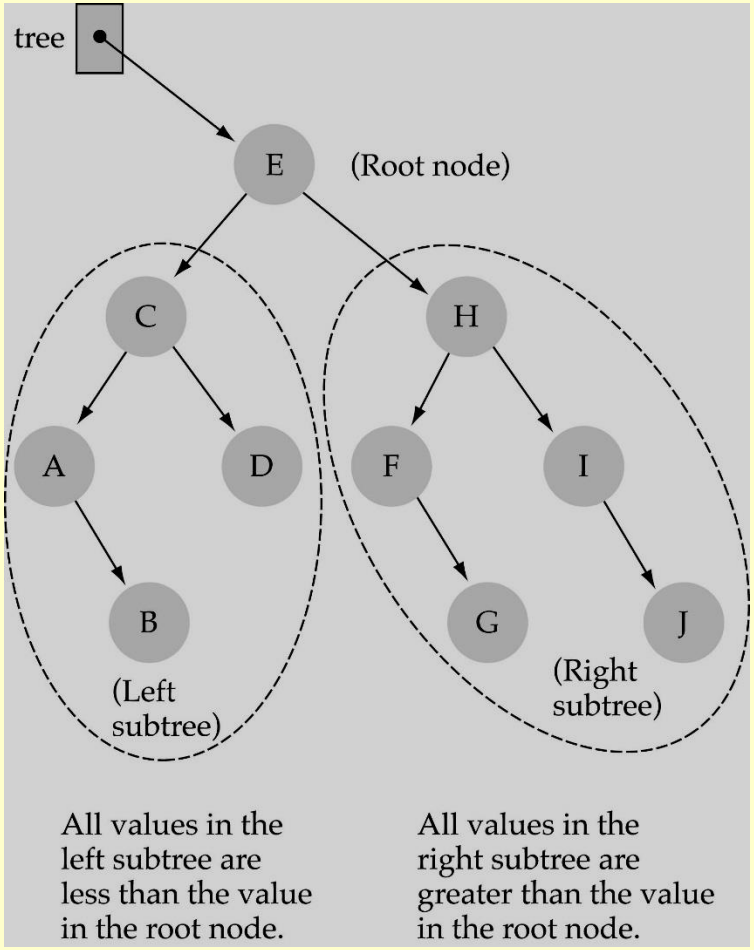


(4) If it is less than the value stored at the root, then search the left subtree

(5) If it is greater than the value stored at the root, then search the right subtree

(6) Repeat steps 2-6 for the root of the subtree chosen in the previous step 4 or 5

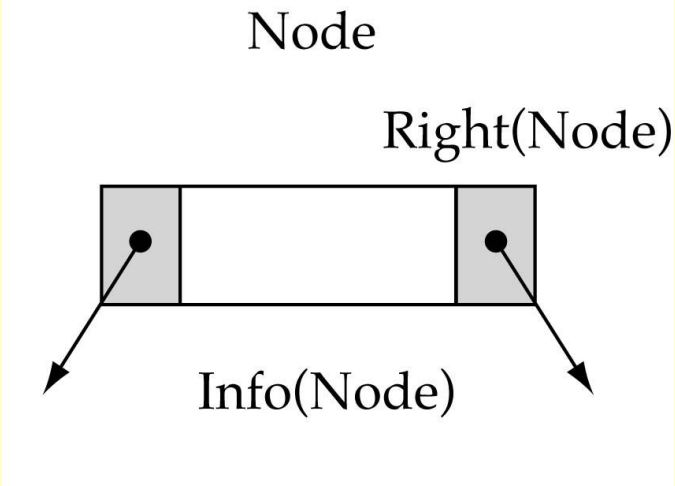
How to search a binary search tree?
查询二叉搜索树



Is this better than searching a linked list?

Yes !! ---> O(logN)

Tree node structure结点结构




```
typedef int data_type;
typedef struct bst_node {
        data_type data;
         struct bst_node *lchild, *rchild;
}bst_t, *bst_p;
```

Binary Search Tree Specification

```
void MakeEmpty();
bool IsEmpty() ;
bool IsFull() ;
int NumberOfNodes();
void RetrieveItem(ItemType&, bool& found);
void InsertItem(data_type);
void DeleteItem(data_type);
void ResetTree(OrderType);
void GetNextItem(data_type &, OrderType, bool&);
void PrintTree(ofstream&);
```

Function NumberOfNodes
计算结点数函数

- Recursive implementation

  #nodes in a tree =

  #nodes in left subtree + #nodes in right subtree + 1

- What is the size factor?

  Number of nodes in the tree we are examining

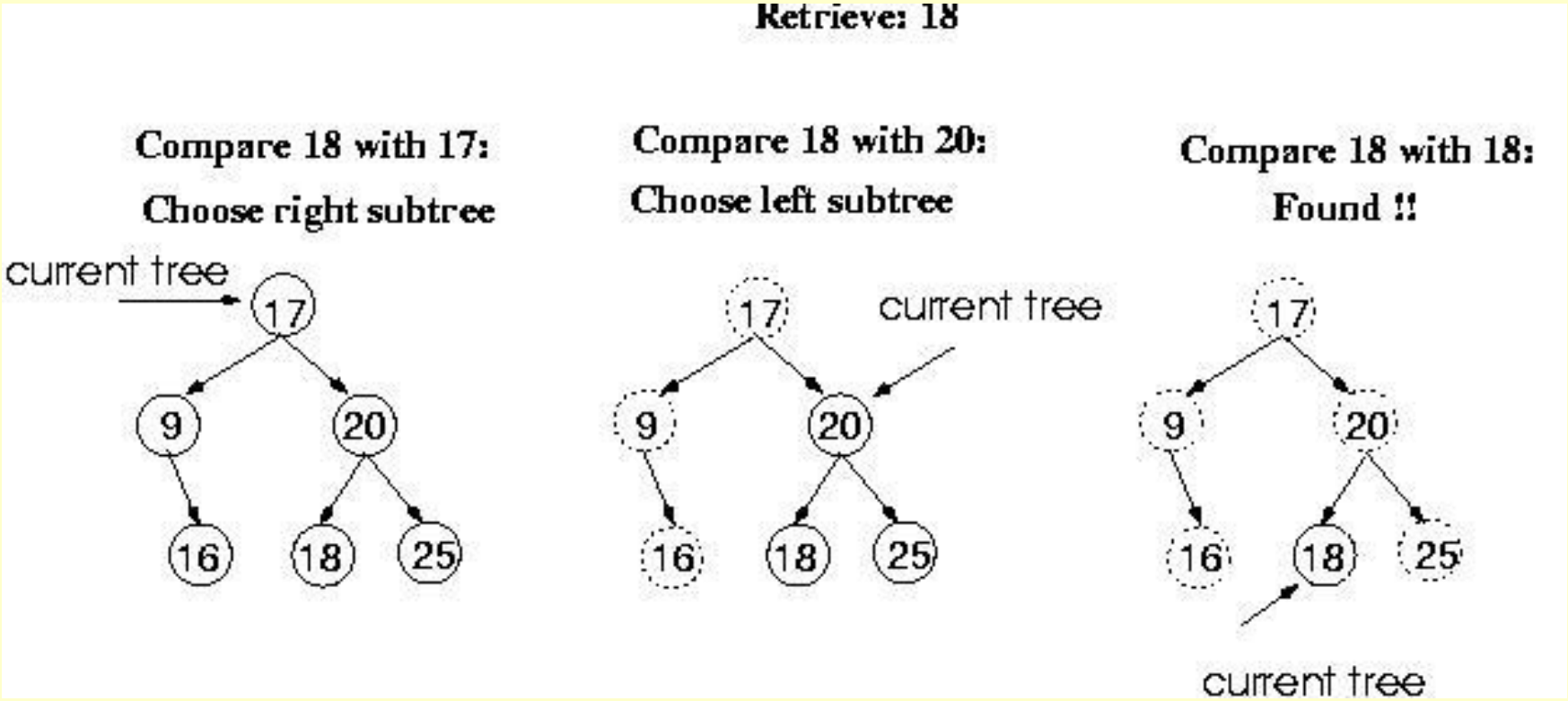
- What is the base case?

  The tree is empty

- What is the general case?

  CountNodes(Left(tree)) + CountNodes(Right(tree)) + 1

Function RetrieveItem



Function RetrieveItem
获取某值结点函数

- What is the size of the problem?

  Number of nodes in the tree we are examining

- What is the base case(s)?

  1) *When the key is found*
  2) *The tree is empty (key was not found)*

- What is the general case?

  Search in the left or right subtrees

Function RetrieveItem (cont.)

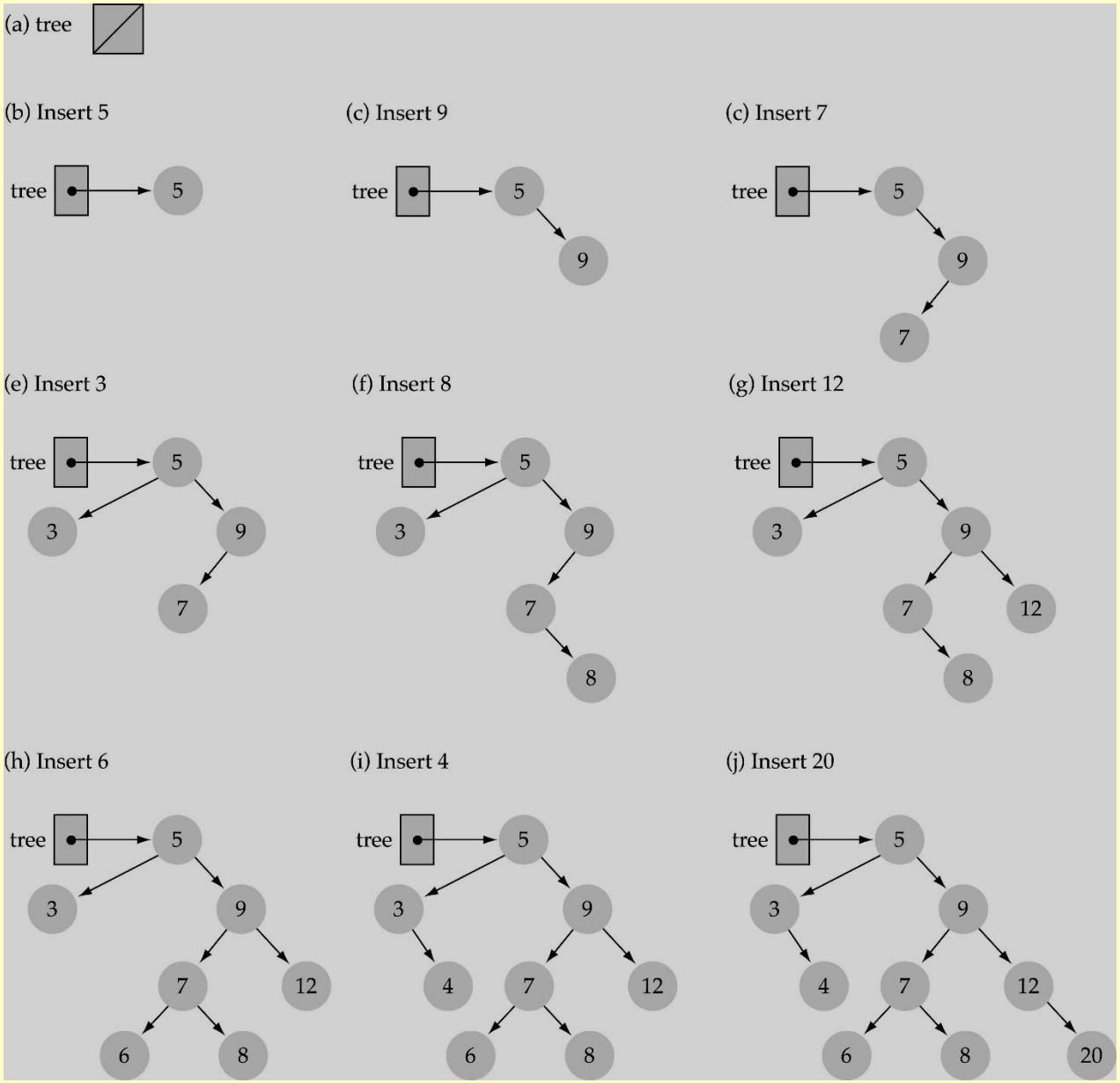
```
int Retrieve ( bst_ p * tree, data_type item )
{
    if (tree == NULL) // base case 2
        return FALSE;
   else if(item < tree->data)
        Retrieve(tree->left, item, found);
   else if(item > tree-> data)
       Retrieve(tree->right, item, found);
   else {  // base case 1
       item = tree-> data;
       return TRUE;
    }
}
```

Running Time?

O(h)

Function InsertItem 插入函数

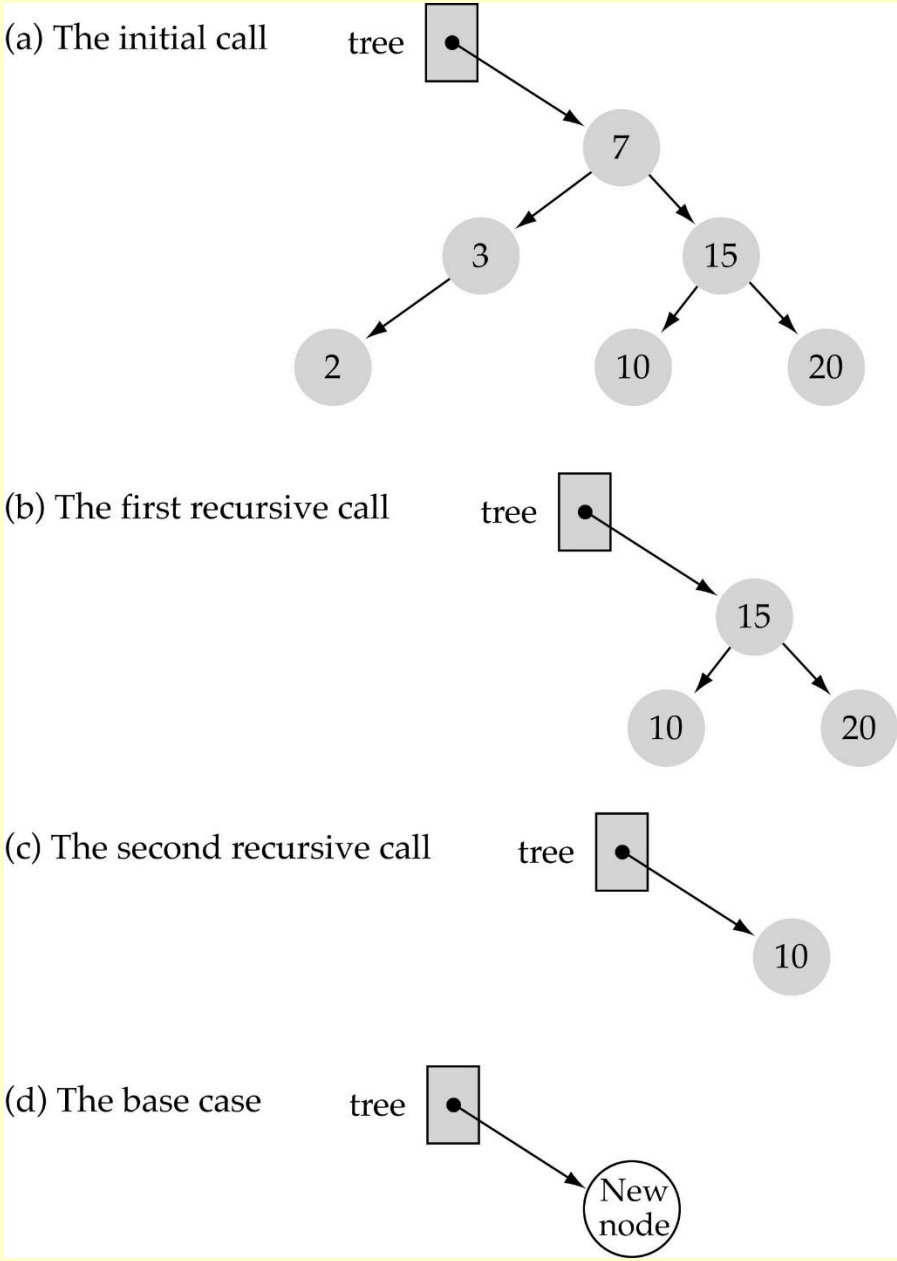
- Use the binary search tree property to insert the new item at the correct place



Function InsertItem (cont.)

- Implementing insertion recursively

e.g., insert 11



Function InsertItem (cont.)
插入函数

- What is the size of the problem?

  Number of nodes in the tree we are examining

- What is the base case(s)?

  The tree is empty

- What is the general case?

  Choose the left or right subtree

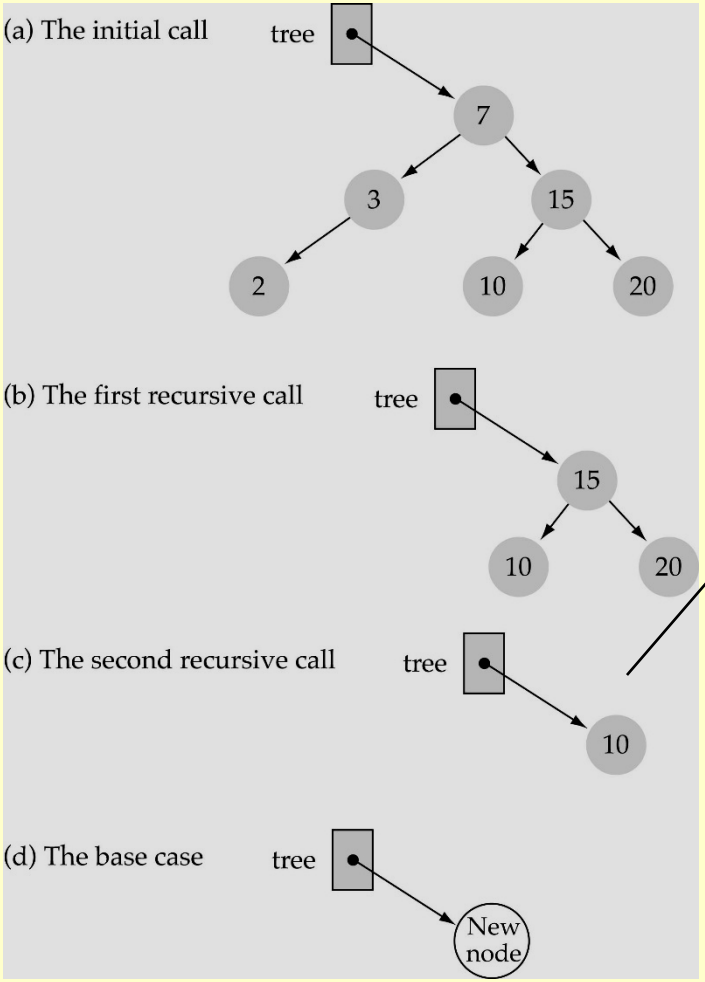
Function InsertItem (cont.)
插入函数

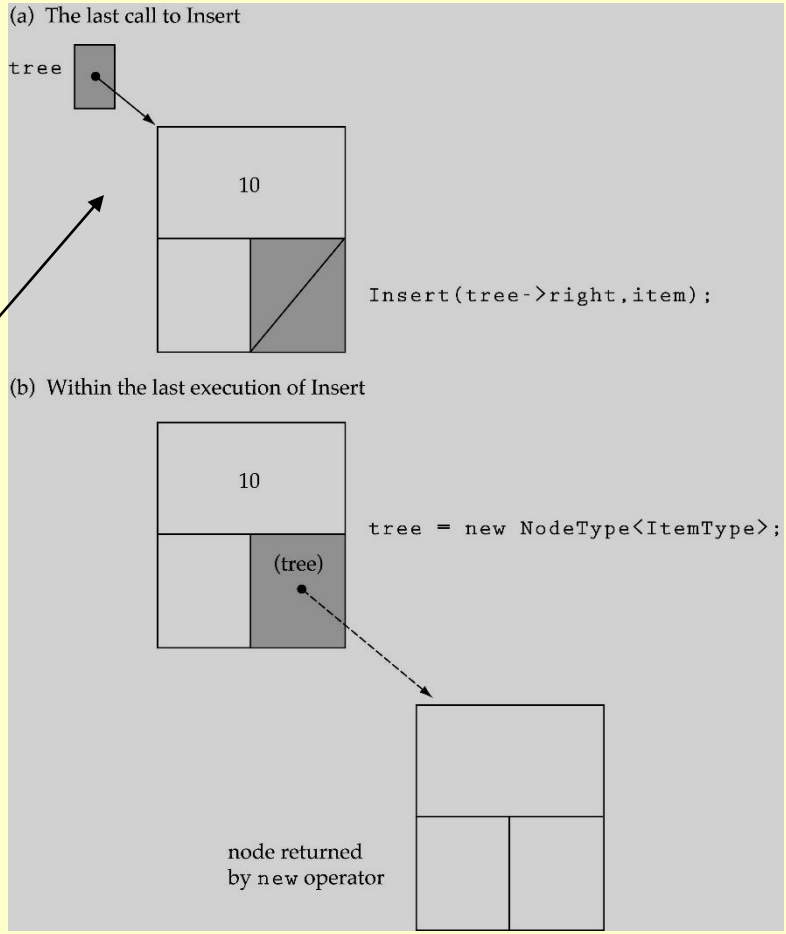
```
void Insert(bst_node *tree, data_type item)
{
    if(tree == NULL) {  // base case
        tree = malloc(sizeof(struct bst_node));
        tree->right = NULL;
        tree->left = NULL;
        tree-> data = item;
     }
    else if(item < tree-> data)
        Insert(tree->left, item);
    else
       Insert(tree->right, item);
}
```

Running Time?

O(h)

Function InsertItem (cont.)
插入函数

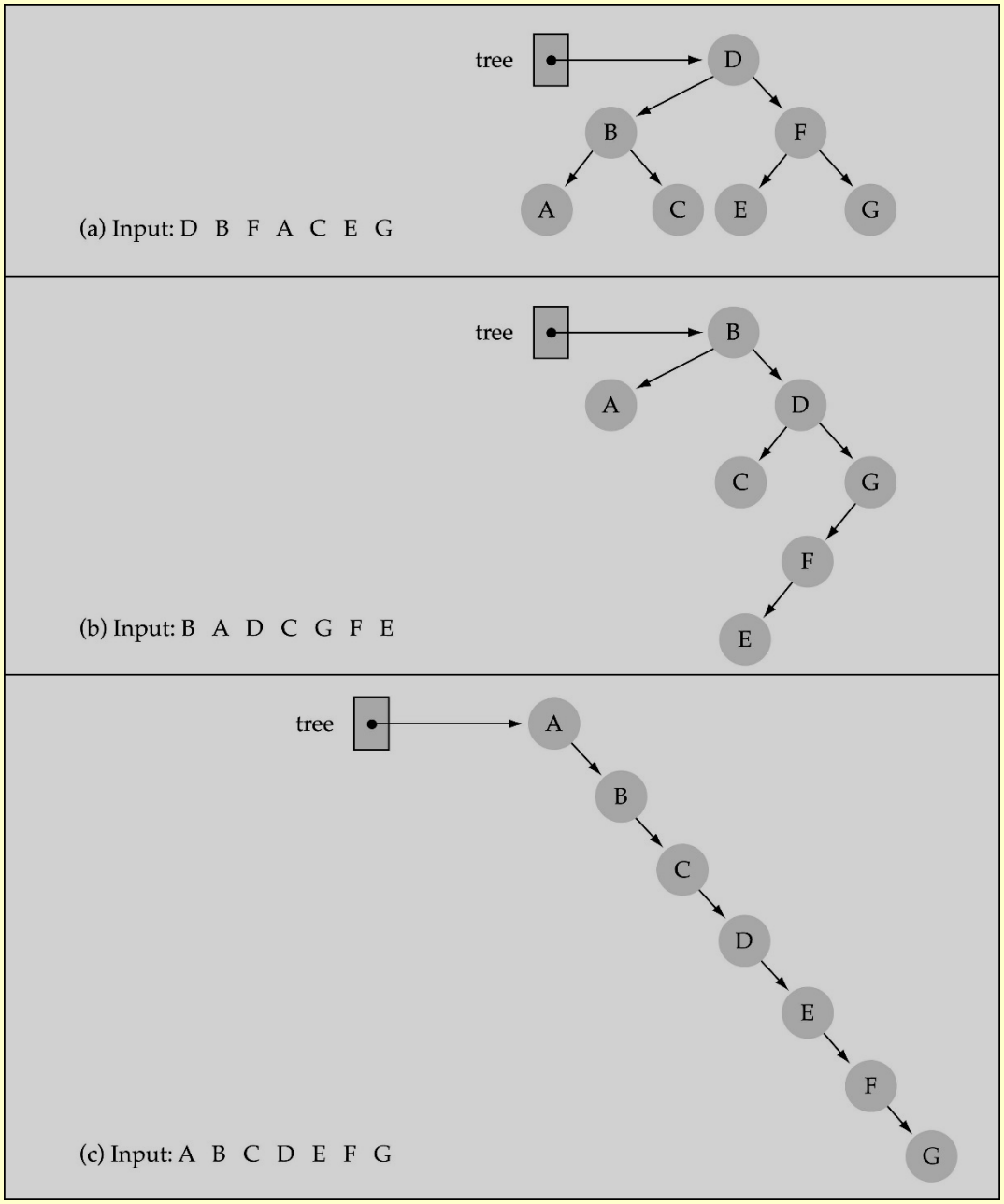




Does the order of inserting elements into a tree matter?
插入次序会影响树的形状

- Yes, certain orders might produce very unbalanced trees!
- 插入元素的次序影响树的性能

Does the order of inserting elements into a tree matter? (cont.)



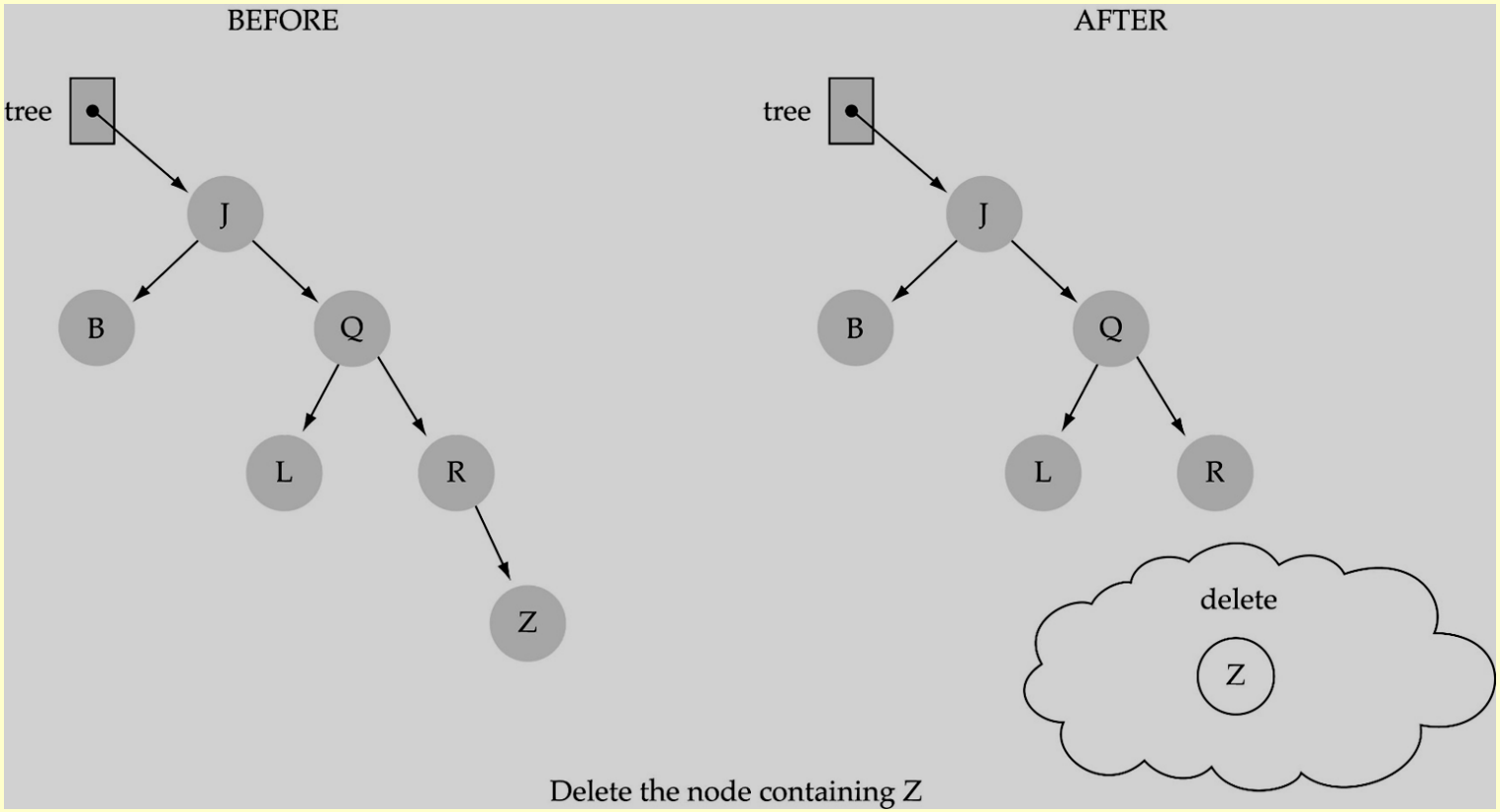
Does the order of inserting elements into a tree matter? (cont'd)

- Unbalanced trees are not desirable because search time increases!不平衡的树
- Advanced tree structures, such as AVL tree, red-black trees, guarantee balanced trees.

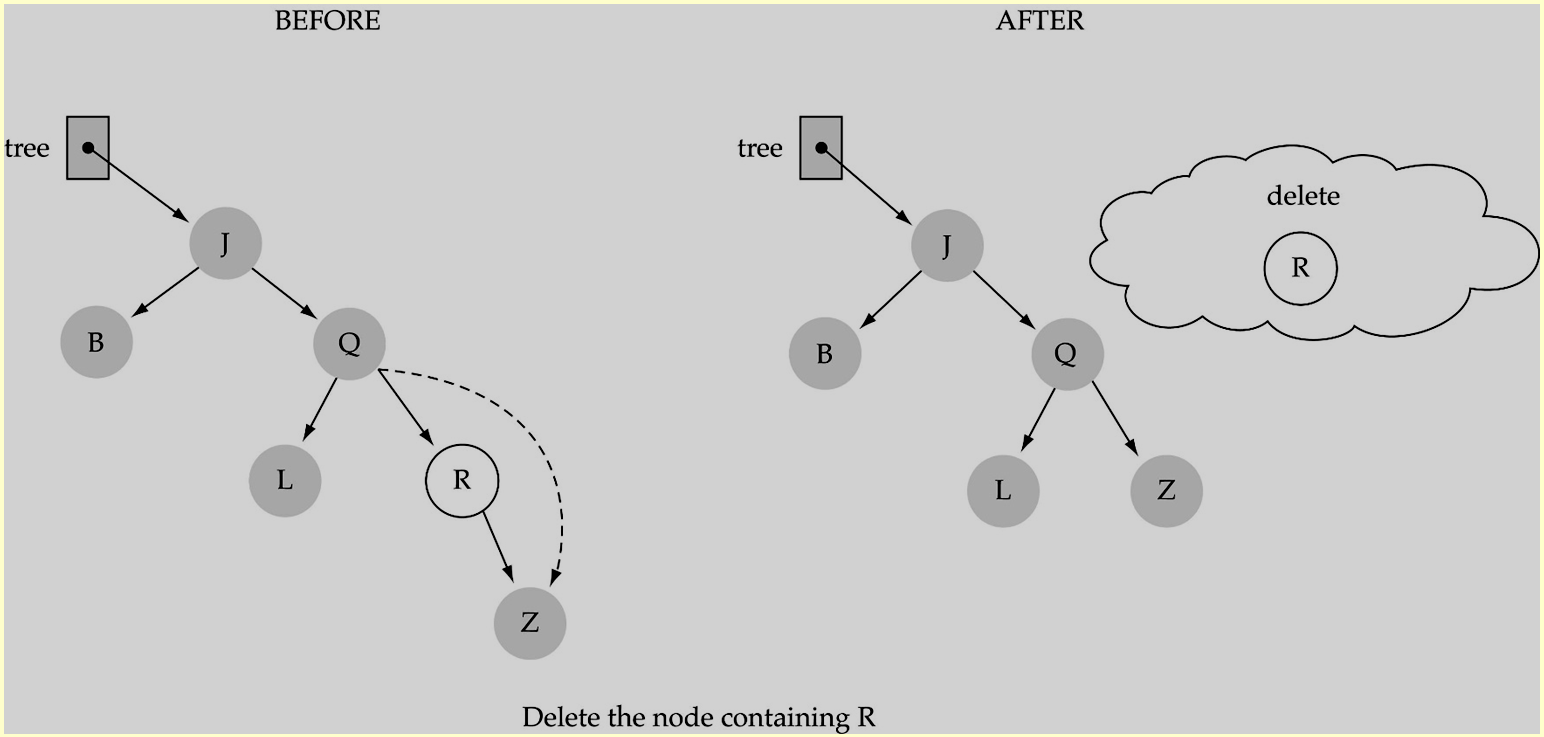
Function DeleteItem
删除函数

- First, find the item; then, delete it
- Binary search tree property must be preserved!!
- We need to consider three different cases:
  - **(1)** Deleting a leaf 删除一个叶结点
  - **(2)** Deleting a node with only one child
    删除有一个孩子结点
  - **(3)** Deleting a node with two children
    删除有两个孩子结点

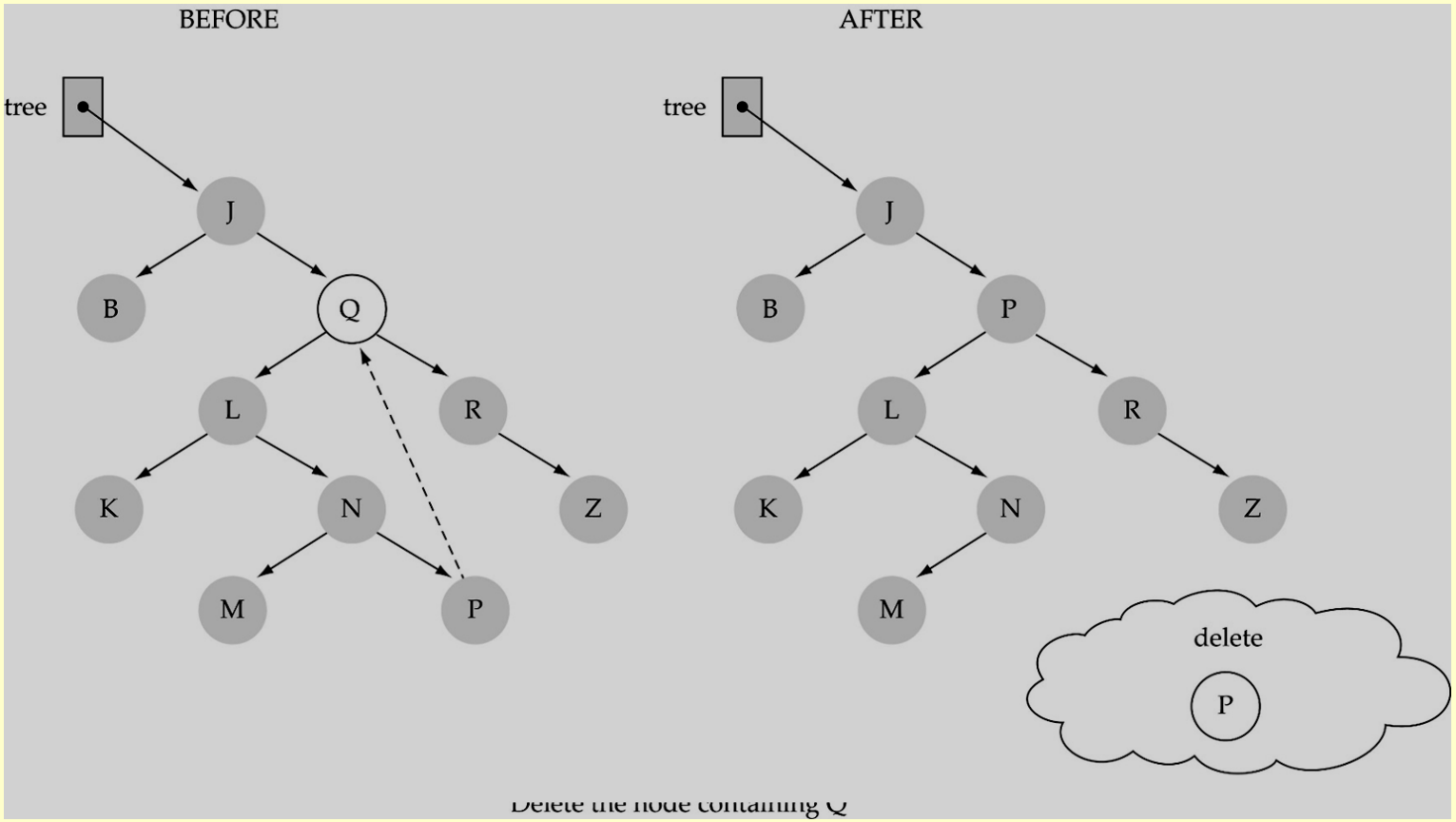
(1) Deleting a leaf 删除叶子结点



(2) Deleting a node with only one child
删除只有一个孩子的结点



(3) Deleting a node with two children
删除有两个孩子的结点



(3) Deleting a node with two children
删除有两个孩子的结点

- Find predecessor (i.e., rightmost node in the left subtree) 找到直接前驱
- Replace the data of the node to be deleted with predecessor‘s data 要删除结点与直接前驱互换
- Delete predecessor node 删除直接前驱结点

Function DeleteItem (cont.)
删除函数

- What is the size of the problem?

  Number of nodes in the tree we are examining

- What is the base case(s)?

  Key to be deleted was found

- What is the general case?

  Choose the left or right subtree

Function DeleteItem (cont.)

```
void Delete (bst_node * tree, data_type item)
{
    if(item < tree->data)
        Delete(tree->left, item);
    else if(item > tree-> data)
        Delete(tree->right, item);
    else
        DeleteNode(tree);
}
```

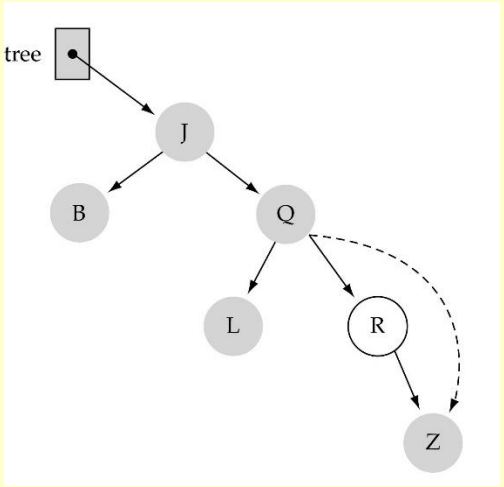
Function DeleteItem (cont.)

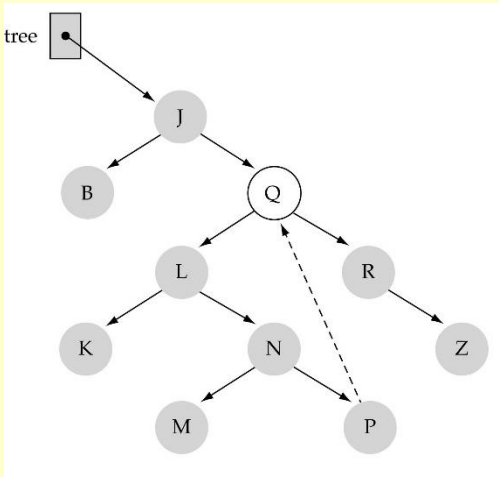
```
void DeleteNode (bst_node * tree)
{
     bst_node  *  tempPtr, prePtr;

    tempPtr = tree;
    if(tree->left == NULL) {    // right child
        tree = tree->right;
        free(tempPtr);
    }
    else if(tree->right == NULL) { // left child
         tree = tree->left;
         free(tempPtr);
    }
    else {
        prePtr=GetPredecessor(tree->left);
        tree与prePtr互换
        free(tempPtr);
    }
}
```

0 children or 1 child

0 children or 1 child

2 children





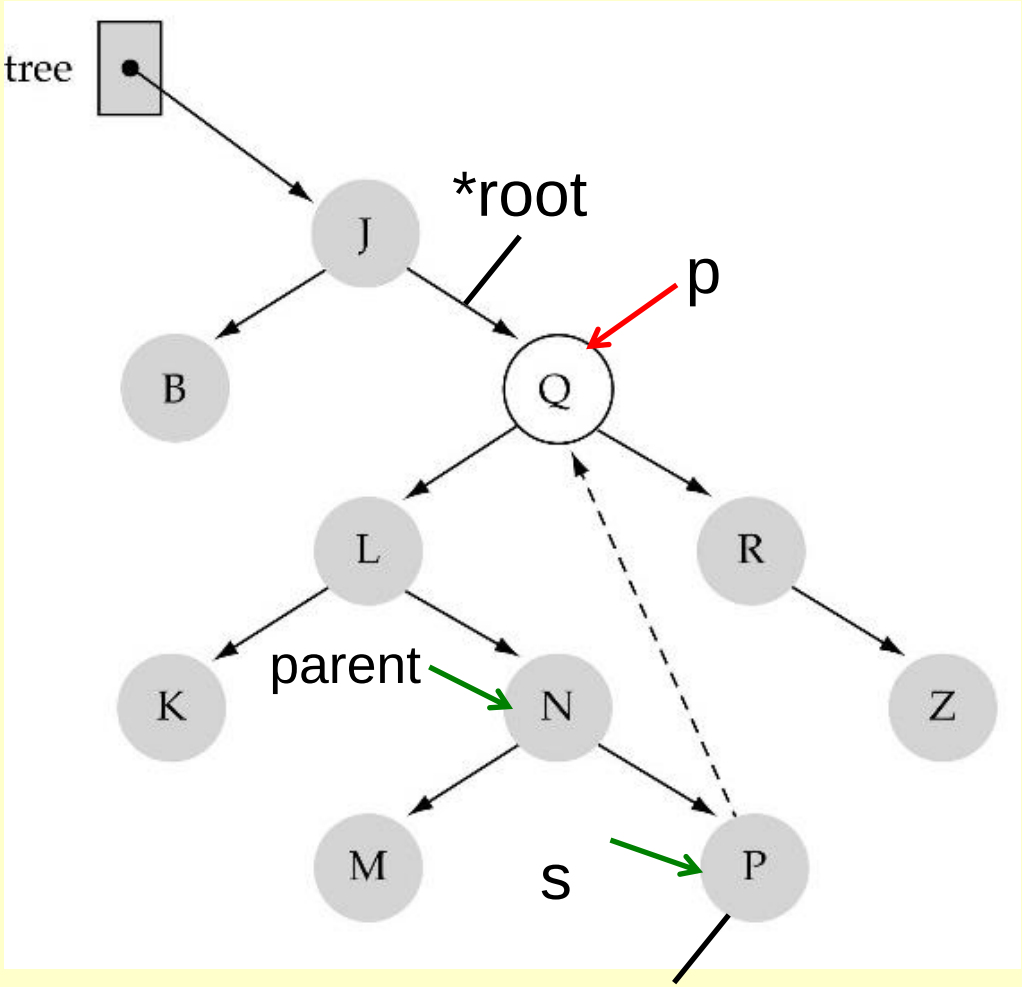
Function DeleteItem (cont.)

```
bst_node * GetPredecessor (bst_node * tree)
{
    while(tree->right != NULL)
        tree = tree->right;
     return tree;
}
```

p与s指向结点互换

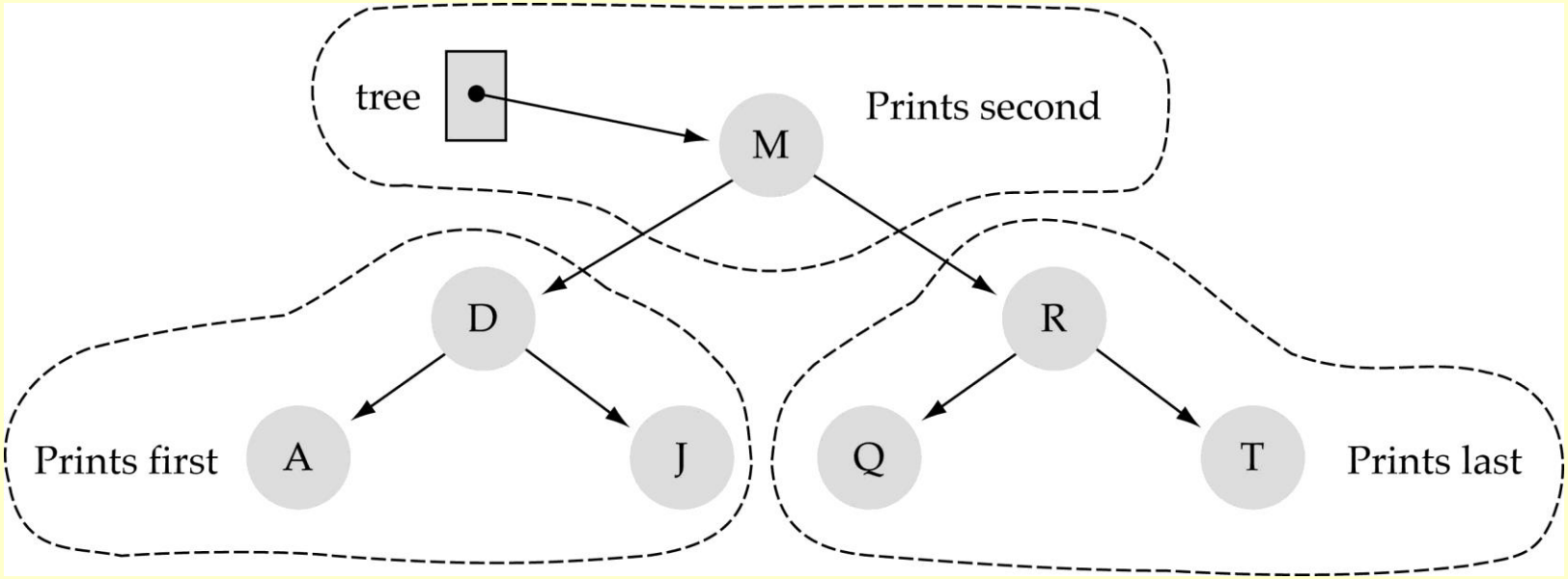
free(p);

与右子树的最左元素互换？6.5



Function PrintTree

- We use "inorder" to print out the node values.
- Keys will be printed out in sorted order.
- Hint: binary search could be used for sorting!



A D J M Q R T

Comparing Binary Search Trees to Linear Lists

Big-O Comparison			
Operation	Binary Search Tree	Array-based List	Linked List
Constructor	O(1)	O(1)	O(1)
Destructor	O(N)	O(1)	O(N)
IsFull	O(1)	O(1)	O(1)
IsEmpty	O(1)	O(1)	O(1)
RetrieveItem	O(logN)*	O(logN)	O(N)
InsertItem	O(logN)*	O(N)	O(N)
DeleteItem	O(logN)*	O(N)	O(N)

*assuming h=O(logN)

Experiment 6.5 Binary Search Tree Implementation

```
Status insert_bst_node(bst_p *root, data_type data);
bst_p search_bst_for_insert(bst_p *root, data_type key);
int Retrieve ( bst_node * tree, data_type item );
int print(data_type data);
int pre_order_traverse(bst_p T);
int in_order_traverse(bst_p T);
int post_order_traverse(bst_p T);
Status delete_bst_node(bst_p *root, data_type data);
```

700.二叉搜索树中的搜索

701.二叉搜索树中的插入操作

450.删除二叉搜索树中的节点

Experiment 6.5 b BST 扩展

98. 验证二叉搜索树：/*中序遍历序列是否有序递归中序遍历将二叉搜索树转变成一个数组，再验证if (vec[i] <= vec[i - 1]) return false; */

530. 二叉搜索树的最小绝对差： /*给一棵所有节点为非负值的二叉搜索树，计算树中任意两节点的差的绝对值的最小值中序遍历二叉搜索树，存储一个有序数组。在一个有序数组上求两个数最小差值*/

501.二叉搜索树中的众数

235. 二叉搜索树的最近公共祖先：/*从上到下遍历，cur节点是数值在[p, q]区间中则说明该节点cur就是最近公共祖先*/

669. 修剪二叉搜索树:/*给定一个二叉搜索树，同时给定最小边界L 和最大边界 R。通过修剪二叉搜索树，使得所有节点的值在[L, R]中 (R>=L) 。 */

108. 将有序数组转换为二叉搜索树

538. 把二叉搜索树转换为累加树:/*给出二叉 搜索 树的根节点，该树的节点值各不相同，请将其转换为累加树（Greater Sum Tree），使每个节点 node 的新值等于原树中大于或等于 node.val 的值之和。 */

1按（）遍历二叉排序树得到是序列是一个有序序列。(易)

A、 先序　　B、 中序 C、 后序　　D、 层次

2、在二排序树中进行查找的效率与（）有关。(易)

A、 二叉排序树的深度　　B、 二叉排序树的结点的个数

C、 被查找结点的度　　D、 二叉排序树的存储结构

3、在常用的描述二叉排序树的存储结构中，关键字值最大的结点（）。(易)

A、 左指针一定为空　　B、 右指针一定为空 C、 左右指针均为空　　D、 左右指针均不为空

4、【2011统考真题】对于下列关键字序列，不可能构成某二叉排序树中一条查找路径的是（）。

A、 95,22,91,24,94,71　　B、 92,20,91,34,88,35 C、 21,89,77,29,36,38　　D、 12,25,71,68,33,34

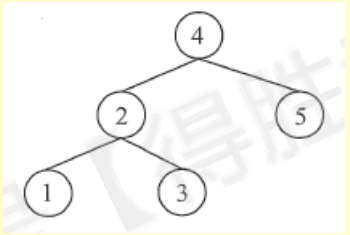
5、从空树开始，依次插入元素52，26，14，32，71，60，93，58，24和41后构成了一棵二叉排序树。在该树查找60要进行比较的次数为（）。

A、 3　　B、 4　　C、 5　　D、 6

6、构造一棵具有n个结点的二叉排序树时，最理想情况下的深度为（）。

A、 n/2　　B、 n　C、 $\log_2(n+1)$ 下取整　　D、 $\log_2(n+1)$ 上取整

7、不可能生成如下图所示的二叉排序树的关键字序列是（）。



A、 4,2,1,3,5　　B、 4,2,5,3,1 C、 4,5,2,1,3　　D、 4,5,1,2,3

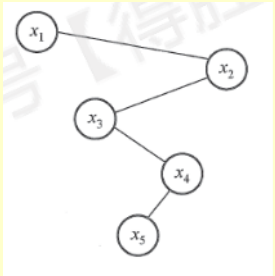
8、设二叉排序树中有n个结点，则在二叉排序树的平均平均查找长度为（ ）。

(A) O(1)　(B) O($\log_2$n)　(C) O(n)　(D) O(n^2)

9、在二叉排序树中插入一个结点的时间复杂度为（ ）。

(A) O(1)　(B) O(n)　(C) O($\log_2$n)　(D) O(n^2)

10、【2018统考真题】已知二叉排序树如图所示，元素之间应满足的大小关系是（）。



A、x1<x2<x5　　B、x1<x4<x5

C、x3<x5<x4　　D、x4<x3<x5